

Matriz de seguridad — contenido HTML/JS en LMS/CMS educativos

Ernesto Serrano · info@ernesto.es

2026-06-14

Índice

1. Tabla resumida (para el cuerpo del artículo)	1
2. Matriz técnica completa (anexo)	2
3. Mitigaciones emparejadas (riesgo → mitigación → limitación)	4
4. Tabla de concienciación (perfil no técnico)	5

Evidencia verificada contra el HEAD de cada repositorio (junio 2026). Cada celda “Cita” remite a `archivo:línea`. Las afirmaciones de comportamiento del navegador están marcadas [hecho] (verificado en código o prueba) o [inferencia].

SHAs de referencia: `mod_exelearning 2c5473d` · `mod_exeweb 60d24fb` · `mod_exescorm e985f4d` · `moodle 2104c372962` (5.x, code en public/) · `exelearning 8101f54e` · `wp-exelearning 9eb07ff` · `omeka-s-exelearning 33faf89` · `wp-franer 7fbf694`.

1. Tabla resumida (para el cuerpo del artículo)

Plataforma / recurso	¿Ejecuta JS del autor?	¿Mismo origen que el LMS?	sandbox del iframe	Aislamiento real
mod_page (Página)	Sí (verificado en vivo: ejecuta <code><script></code>)	Sí	— (no es iframe)	Ninguno server-side (<code>noclean</code>); gated por capacidad (<code>RISK_XSS</code>)
mod_scorm (core)	Sí	Sí	ninguno	Ninguno (confía en el SCO)
mod_h5pactivity / core_h5p	Parámetros No / librerías Sí (<code>preloadedJs</code>)	Sí (<code>about:blank</code> hereda origen)	— (<code>contentDocument.write</code> , sin sandbox)	Parámetros filtrados por semántica; el JS de librería corre <i>same-origin</i> , gate <code>h5p:updatelibraries</code>
mod_exelearning	Sí	Sí	<code>allow-scripts</code> <code>allow-same-origin</code> <code>allow-popups allow-forms</code> <code>allow-popups-to-escape-sandbox</code>	Parcial (bloquea top-nav y modals)
mod_exeweb	Sí	Sí	ninguno	Ninguno
mod_exescorm	Sí	Sí	ninguno	Ninguno
wp-exelearning	Sí	Configurable: secure opaco (def.) / legacy same-origin	<code>secure: allow-scripts</code> <code>allow-popups · legacy: +</code> <code>allow-same-origin</code>	Fuerte en secure (origen opaco); parcial en legacy
omeka-s-exelearning	Sí	Configurable: secure opaco (def.) / legacy same-origin	<code>secure: allow-scripts</code> <code>allow-popups</code> <code>allow-popups-to-escape-sandbox</code> <code>· legacy: +</code> <code>allow-same-origin</code>	Fuerte en secure (origen opaco); ahora también abre vistas públicas
wp-franer (referencia)	Sí, aislado	No (srcdoc opaco)	<code>allow-scripts</code> <code>allow-forms + CSP</code> inyectada	El más fuerte

Lectura rápida: *ejecutar JavaScript del autor no es el problema; el problema es ejecutarlo **con el mismo origen** que el LMS y **sin** una frontera explícita*. Las dos columnas que de verdad importan son “mismo origen” y “aislamiento real”.

2. Matriz técnica completa (anexo)

2.1 *mod_exelearning* (2c5473d)

Atributo	Valor	Cita
Cómo se publica	El profesor sube un .elpx; se extrae y se sirve por <code>pluginfile.php</code>	<code>lib.php:513-568</code>
HTML/JS arbitrario	Sí (HTML/JS del autor del paquete)	<code>view.php:446-447</code>
JS se ejecuta	Sí	<code>iframe allow-scripts</code>
Mismo origen	Sí (<code>\$CFG->wwwroot</code> único)	<code>view.php:446-447</code>
Usa <code>iframe</code>	Sí, <code>#exelearningobject</code>	<code>view.php:446-447</code>
<code>sandbox</code>	<code>allow-scripts allow-same-origin allow-popups allow-forms allow-popups-to-escape-sandbox</code>	<code>view.php:446-447</code>
<code>allow-top-navigation</code>	No (omitido a propósito)	<code>view.php:446-447</code>
<code>allow-popups-to-escape-sandbox</code>	Sí (residuo a quitar)	<code>view.php:446-447</code>
CSP	No emitida	<code>lib.php:513-568</code>
Permissions-Policy	No emitida (pendiente)	<code>lib.php:513-568</code>
X-Frame-Options	No (en <code>pluginfile</code>); core pone <code>sameorigin</code> en páginas	<code>weblib.php:1604-1605</code>
<code>postMessage</code>	Sí, solo en el editor (no en el visor)	<code>amd/src/editor_modal.js:441-449</code>
Valida <code>event.origin</code>	Sí (editor)	<code>editor_modal.js:441-449</code>
Valida <code>event.source</code>	Sí (<code>=== iframe.contentWindow</code>)	<code>editor_modal.js:441-449</code>
Token/contexto	<code>sesskey</code> validado server-side en <code>track.php</code>	<code>track.php:40</code> , <code>view.php:387-390</code>
Acceso a <code>parent</code>	Sí (mismo origen)	bridge SCORM
Acceso a cookies no-HttpOnly	Sí (mismo origen)	mismo origen
Acceso al <code>sesskey</code>	Sí (va en la URL de <code>track.php</code> , legible desde el DOM/JS)	<code>view.php:387-390</code>
Localiza formularios edición	Sí, si el DOM padre los tiene	mismo origen
Cambios reales	No demostrado (server valida <code>sesskey</code> + capability)	<code>track.php:40</code>
Mitigaciones	<code>Sandbox</code> parcial; <code>require_capability</code> en <code>pluginfile</code> ; <code>require_sesskey</code> server-side	<code>lib.php:513-568</code> , <code>track.php:40</code>
Riesgos que quedan	XSS cross-component same-origin (riesgo residual)	—
Impacto de IA sin revisar	Alto: JS generado se ejecuta con el origen de Moodle	—

Dependencias **duras** de same-origin (impiden quitar `allow-same-origin` sin reescribir el bridge): 1. El padre lee `iframe.contentDocument` para mapear `objectid` → `js/scorm_tracker.js:75-86` y `:151-154`. 2. El hijo (pipwerks) recorre `window.parent` buscando `window.API` → `assets/scorm/SCORM_API_wrapper.js:62-118`. 3. El *teacher-mode hider* inyecta CSS en `contentDocument` → `classes/local/ui/teacher_mode_hider.php:44-61`.

2.2 *mod_exeweb* (60d24fb) y *mod_exescorm* (e985f4d)

Atributo	<i>mod_exeweb</i>	<i>mod_exescorm</i>
<code>iframe</code>	<code>embed_general.mustache:32-34</code>	<code>player.php:283-285</code> (noscript)
<code>sandbox</code>	ninguno	ninguno
HTML/JS arbitrario	Sí	Sí
Mismo origen	Sí	Sí
SCORM API walk	No (sin tracking)	Sí (<code>SCORM_API_wrapper.js:71-78,140-142</code>)
<code>sesskey</code>	—	URL param (<code>datamodels/scorm_12.js:19-24</code>)
CSP en <code>pluginfile</code>	No (<code>lib.php:448-512</code>)	No (<code>lib.php:1064-1142</code>)
<i>teacher-mode hider</i>	Sí (<code>locallib.php:90-107</code>)	—

2.3 Moodle core: *mod_scorm* (2104c372962)

Atributo	Valor	Cita
<code>iframe</code> del SCO	sin sandbox	<code>public/mod/scorm/player.php:279-285</code>
API discovery	<code>myFindAPI</code> recorre <code>win.parent</code> y <code>window.opener</code>	<code>loadSCO.php:113-122, :128-129</code>
<code>sesskey</code>	<code>confirm_sesskey()</code> antes de guardar	<code>datamodel.php:53</code>
Capability	<code>mod/scorm:savetrack</code>	<code>datamodel.php:56</code>
Mismo origen	Sí (<code>wwwroot/pluginfile.php</code>)	<code>locallib.php:2323,2327</code>

Veredicto: *mod_scorm* corre HTML/JS no confiable **sin sandbox** — más débil que *mod_exelearning*. Su defensa es server-side (`sesskey` + capability), no aislamiento del cliente.

2.4 Moodle core: core_h5p / mod_h5pactivity (2104c372962)

Atributo	Valor	Cita
iframe	src="about:blank"	h5piframe.mustache:33
Construcción	contentDocument.write(...)	h5p.js:390-394
Origen del contenido	hereda el del padre (same-origin) [hecho]	h5p.js:391-394 (nota verificada)
HTML/JS arbitrario (parámetros)	No: content.json validado contra la semántica; filter_xss sin <script> ni on*	h5p.classes.php:2260-2282 (filterParameters), :4303-4384, :5033-5054
HTML/JS de librería (preloadedJs)	Sí: código de confianza, <script src=pluginfile.php/.../core_h5p> same-origin y sin sandbox	player.php:484-500, h5p.js:391-437
Gate de instalación de librería	moodle/h5p:updatelibraries (arquetipo manager, RISK_XSS), evaluada contra quien sube; profesorado solo h5p:deploy	lib/db/access.php, helper.php:210-224, api.php:403-405, h5p.classes.php:1577-1579
Whitelist	Extensiones curadas (content + librería)	framework.php:698-700
postMessage handler	Valida event.source===window.parent y event.data.context==='h5p'; no valida event.origin	embed.js:38-46, h5p.js:457-465
postMessage envío	targetOrigin = '*' (origen-agnóstico)	embed.js:64-73, h5p.js:484-493
\$CFG->h5pcrossorigin	CORS para media (img/audio/video), no para el iframe	helper.php:383, config-dist.php:778-786

Veredicto (matizado): hay que distinguir dos planos. En los **parámetros** de contenido, H5P **no ejecuta HTML/JS de autor**: filtra el texto con `filter_xss` contra una lista cerrada de etiquetas sin `<script>` y descarta `on*` (h5p.classes.php:4303-4384, :5033-5054) — control negativo, más controlado por diseño que un .elpx. En el plano de las **librerías**, en cambio, **sí**: el `preloadedJs` de una librería es **código de confianza** que corre *same-origin* y sin sandbox (player.php:484-500, h5p.js:391-437); la barrera para que el autor introduzca su propia librería es la **capacidad** moodle/h5p:updatelibraries (gestión/administración, RISK_XSS), no el saneamiento — **mismo patrón que mod_page**. PoC: poc/evil-h5p-library.h5p; evidencia: evidencias/resultados-h5p-library.json. Además **no es inmune** por contenido: XSS documentados (MDL-67110, CVE-2024-43439 —XSS reflejado vía mensaje de error—, CVE-2024-3111 —stored XSS vía SVG—) y precedente de RCE por import de .h5p (CVE-2026-30875, Chamilo). Su `postMessage` no valida `event.origin` (mitiga con `event.source + context`), patrón a no copiar a ciegas.

Acción privilegiada (Moodle, verificado en código): el contenido same-origin (Página, .elpx, SCO) lee `window.M.cfg.sesskey` y puede invocar servicios web **AJAX** como `core_user_update_users` ('ajax' => true, cap moodle/user:update, public/lib/db/services.php:1982-1989) → si lo abre un usuario privilegiado, puede forjar la edición de una cuenta. La defensa es server-side (capacidad + validación), no ocultar el token.

2.5 Moodle core: mod_page + filtrado global (2104c372962)

Atributo	Valor	Cita
Render	format_text(..., noclean=true)	public/mod/page/view.php:90-93
trusttext por defecto	0 (desactivado)	locallib.php:53
¿trusttext activo?	!empty(\$CFG->enabletrusttext)	weblib.php:958-961
¿texto confiable?	activo y capability	weblib.php:949-950
Capability	moodle/site:trustcontent (RISK_XSS, solo profesor/manager)	db/access.php:452-454
noclean en salida	\$formatoptions->noclean = true	public/mod/page/lib.php:352
purify_html (otros contextos)	HTMLPurifier quita <script> en texto no noclean	weblib.php:1105-1107
enabletrusttext por defecto	0 (desactivado)	public/admin/settings/security.php:68
X-Frame-Options	sameorigin salvo \$CFG->allowframembedding	weblib.php:1604-1605
CSP global	ninguna por defecto	weblib.php (NOT FOUND)

Veredicto (verificado en vivo): en `mod_page`, `<script>` y `<img onerror>` se ejecutan — `noclean=true` hace que `format_text` traduzca a `clean=false` y **no** pase por `HTMLPurifier`. El filtrado `purify_html` aplica a otros campos de texto no confiable (sin `noclean`), **no** a la salida de `mod_page`. La protección de `mod_page` es la **capacidad** de crear/editar la Página (`mod/page:addinstance`; HTML confiable ligado a `moodle/site:trustcontent`, RISK_XSS), no el saneado. `enabletrusttext=0` por defecto **no** impide la ejecución porque `mod_page` usa `noclean`.

2.6 eXeLearning core / contenido exportado (8101f54e)

Atributo	Valor	Cita
SCORM API walk	<code>window.parent (500) + win.top.opener</code> , sin validación de origen	<code>SCORM_API_wrapper.js:71-77, :136-141</code>
localStorage	clave <code>exeTeacherMode</code> (revela contenido de profesor)	<code>exe_export.js:100-130</code>
postMessage listener	valida <code>type</code> , no <code>event.origin</code>	<code>asset_url_resolver.js:905-906</code>
<code>eval()</code>	sí (callbacks de carga de script, tooltips, onclick)	<code>common.js:805,810,757;</code> <code>common_edition.js:39</code>
MathJax	local empaquetado (no CDN)	<code>common.js:26-115</code>
iframes exportados	sin sandbox	<code>asset_url_resolver.js:933-936</code>
Validación server-side	ninguna (HTML exportado es estático/self-contained)	—

2.7 wp-exelearning (9eb07ff)

Atributo	Valor	Cita
Modo iframe	ajuste <code>exelearning_iframe_sandbox_mode: secure (def.) / legacy</code> ; helper único, <i>fail-safe</i> a <code>secure</code>	<code>includes/class-iframe-sandbox.php</code>
iframe (secure)	<code>sandbox="allow-scripts allow-popups"</code> (sin <code>same-origin</code> → opaco) + <code>referrerpolicy="no-referrer"</code>	<code>public/class-shortcodes.php (sandbox_tokens())</code>
iframe (legacy)	<code>sandbox="allow-scripts allow-same-origin allow-popups"</code> (<code>same-origin</code>)	<code>class-iframe-sandbox.php (TOKENS_LEGACY)</code>
Teacher mode (secure)	server-side por la <code>src</code> (<code>exe-teacher/exe-teacher-toggler</code>), aplicado por el content proxy	<code>includes/class-content-proxy.php</code>
Content proxy	<code>permission_callback => '__return_true'</code> (lectura no autenticada por hash SHA1)	<code>includes/class-exelearning-rest-api.php:44-50</code>
Nonce en guardado	usa <code>permission_callback</code> (capability), no <code>wp_verify_nonce</code>	<code>rest-api.php:223</code>
Nonce en editor	<code>wp_verify_nonce</code> al cargar página	<code>class-exelearning-editor.php:111</code>

2.8 omeka-s-exelearning (33faf89)

Atributo	Valor	Cita
Modo iframe	ajuste <code>exelearning_iframe_mode: secure (def.) / legacy</code> ; helper único, <i>fail-safe</i> a <code>secure</code>	<code>src/Service/IframeSandbox.php</code>
iframe (secure)	<code>sandbox="allow-scripts allow-popups allow-popups-to-escape-sandbox"</code> (sin <code>same-origin</code> → opaco); también en las dos vistas públicas (antes fijadas a <code>allow-same-origin</code>)	<code>ExeLearningRenderrer.php, view/.../public/*-show.phtml</code>
iframe (legacy) <code>src</code>	añade <code>allow-same-origin</code> fijado por JS desde <code>window.location</code> (prefijo de base)	<code>IframeSandbox::tokens()</code> <code>ExeLearningRenderrer.php</code>
CSRF	token obligatorio (rechaza vacío/nulo)	<code>src/Controller/CsrfValidationTrait.php:24-38</code>
postMessage	mixto: <code>editor.js:83</code> usa <code>'*'</code> ; <code>omeka-exe-download.js:122-125</code> y <code>omeka-exe-bridge.js:46</code> usan origen específico	<code>cit</code>

2.9 wp-franer (7fbf694) — patrón de referencia

Atributo	Valor	Cita
iframe	<code>srcdoc + sandbox="allow-scripts allow-forms"</code> (sin <code>same-origin</code> /popups)	<code>public/partials/franer-public-render.php:141-148</code>
CSP inyectada	<code>default-src 'none'; ... connect-src 'none'; form-action 'none'; base-uri 'none'</code>	<code>includes/class-franer-sanitizer.php:48</code>
Escapado <code>srcdoc</code>	<code>_wp_specialchars(..., double_encode=true)</code>	<code>sanitizer.php:150-154</code>
Validación mensaje	<code>event.source</code> contra <code>contentWindow</code> conocido	<code>public/js/franer-shell.js:89-100, :115-117, :344-355</code>
Fetch autenticado	lo hace el padre , con <code>X-WP-Nonce</code>	<code>franer-shell.js:288-295</code>

3. Mitigaciones emparejadas (riesgo → mitigación → limitación)

Riesgo	Mitigación	Qué no resuelve
Lectura de cookies de sesión	HttpOnly + SameSite=Strict	No protege frente a XSS en el mismo origen ni a tokens expuestos en el DOM
Acceso al parent / mismo origen	iframe de origen opaco sin allow-same-origin (o srcdoc , o subdominio)	Rompe el bridge SCORM directo (window.API); exige bridge postMessage
Localización/envío de formularios con sesskey /nonce	Validación server-side del token por acción + capacidades mínimas	Si el server confía en el cliente, ocultar el token en el DOM no sirve
postMessage no validado	Allowlist cerrada + validación de event.origin y event.source	Una allowlist mal mantenida reabre la superficie
Ejecución de JS arbitrario	CSP restrictiva + sandbox sin allow-same-origin	El contenido legítimo que necesita JS (SCORM/H5P) exige arquitectura de bridge explícita
Navegación superior / popups	Omitir allow-top-navigation y allow-popups-to-escape-sandbox	Puede degradar UX de contenido legítimo que abre recursos externos

Principio rector: la mitigación efectiva vive en el **servidor** y en las **cabeceras**, no en confiar en que el cliente “no encuentre” el token.

4. Tabla de concienciación (perfil no técnico)

Riesgo detectado	Por qué importa	Mitigación	Limitación de la mitigación
El recurso puede leer la cookie de sesión	Permitiría suplantar al usuario autenticado	HttpOnly + SameSite=Strict	No cubre XSS en el mismo origen
El recurso accede al marco padre	Acceso al DOM autenticado del LMS	Origen opaco sin allow-same-origin	Rompe SCORM directo; exige bridge
El recurso localiza el sesskey /nonce	Primer paso para forzar acciones	Validación server-side por acción	Inútil si el server confía en el cliente
El recurso localiza formularios de edición	Permitiría crear/modificar contenido	Capacidades mínimas + validación server-side	Depende de la configuración de roles
postMessage sin validar origen/source	Canal de control no autenticado	Allowlist + validación origin/source	Allowlist mal mantenida reabre el hueco